# Graph drawing by stress minimization

Laslo Hauschild, Lennart Peters

March 29, 2026

## Contents

**1 Introduction** **3**

**2 Graph embedding** **3**
2.1 Graph embedding . . . 3
2.2 Modeling stress . . . 4
2.2.1 FR model . . . 4
2.2.2 ERLL model . . . 4

**3 Implementation** **5**
3.1 Plotting a graph embedding . . . 5
3.1.1 Higher dimension plotting . . . 5
3.2 Stress functions . . . 6
3.3 Gradient descent . . . 7
3.3.1 Simple gradient descent . . . 7
3.3.2 Nesterov's accelerated gradient descent . . . 8

**4 Results** **10**
4.1 Figuring out good learning rates . . . 10
4.1.1 Simple gradient descent, FR stress . . . 10
4.1.2 Simple gradient descent, ERLL stress . . . 13
4.2 Some graphs . . . 14
4.2.1 Icosahedral . . . 15
4.2.2 Karate network . . . 16
4.2.3 Dorogovtsev Mendes Graph . . . 17
4.2.4 Circular Ladder Graph . . . 18
4.2.5 Watts Strogatz Graph . . . 19
4.2.6 Static Scale Free Graph . . . 20
4.3 Conclusion . . . 21

**5 References** **21**

# 1 Introduction

Graphs are a prominent object in modern mathematics. They are quite useful for abstracting complex reallife situations, e.g. railway networks, data structure, geometric objects, etc. Imagine two companies. For each company, employees are denoted as vertices, and daily interactions between employees as edges. A researcher could now compare the companies on communication through hierarchy, between employees of the same department, etc., by studying the two graphs. The researcher needs to meticulously search through huge lists of vertices and edges. Having a graph be represented as a simple drawing, vertices as points, and edges as straight-line segments, would speed up the process. That is part of our goal. We want a lossless representation of finite graphs in $\mathbb{R}^2$ and $\mathbb{R}^3$ in an esthetically pleasing way s.t. we can quickly gain a better understanding of what type a given graph is.

# 2 Graph embedding

A graph $G = (V, E)$ is an ordered pair of vertices $V$ and a set of edges $E \subseteq \{\{u, v\} \mid u, v \in V\}$, an edge $e$ being $e = \{u, v\}$ . Since our ultimate goal is to have a drawn version of a given graph $G$, it is natural to limit ourselves to a finite set $V$. A finite set $V$ induces $E$ to be a finite set. First, we will define a general way to embed graphs in vectorspaces $\mathbb{R}^n$. We will see that every graph $G$ has uncountably many possible embeddings $H$ and then optimize the look of an embedding to a given graph $G$.

## 2.1 Graph embedding

Let $G$ be a graph. We define a map as $p : V \to A$, which maps every vertex $v \in V$ to a point $a \in A$. Let us call the image $W = Im(p) \subseteq A$ and consider $W$ as a set of vertices, then we get the following.

$$p \quad \text{is injective} \iff \text{There exists exactly one bijection} \quad p = q : V \to W \tag{1}$$

Let $\Omega = \{\{a, b\} \mid a, b \in A\}$ be the set of all unordered pairs of $W$. We define a function $P : E \to \Omega, \{u, v\} \mapsto \{p(u), p(v)\}$ and call $F = Im(P) \subseteq \Omega$. Since $p$ is injective, therefore $P$ is injective and by the same argument we get the following.

$$P \quad \text{is injective} \iff \text{There exists exactly one bijection} \quad P = Q : E \to F \tag{2}$$

This means, an injective $p$ induces (1) and (2). For the existence of such a $p$, we need to have

$$|V| \leq |A|. \tag{3}$$

Gathering what we have, if (3) is true, then there exist embeddings $p$, then $H = (p(V), P(E))$ is equivalent to $G$. To keep it short, let us say $H = p(G)$.

## 2.2 Modeling stress

We now choose $A$ to be an $\mathbb{R}$-vectorspace, s.t. we can benefit from their analytic properties. For $n \geq 1$, an injective $p : V \to \mathbb{R}^n$ induces an embedding of $G$ in $\mathbb{R}^n$, but we add a drawing function $d : F \to \mathbb{R}^n, \{a, b\} \mapsto \overline{ab}$ that draws an edge as a straight-line segment. Drawing $H$ this way results in a bounded interval in $\mathbb{R}$, where straight-line segments overlap with the points, which we deem unfit for representation purposes. For $n \geq 4$, we draw in high dimensions and we will see, that $n = 2, 3$ suffice. When embedding a graph $G$ into $\mathbb{R}^n$, we essentially choose a finite set of points $W$. For every graph $G$ there exist uncountably many embeddings into $\mathbb{R}^n$. We want to gain as much order as possible. Graphs can end up looking tangled, because they are randomly chosen points by construction. A broadly used approach is to use a stress-function $S$ that assigns the graph $H$ a real number, the stress in $H$. The local minima of $S$ are our desired representations of $G$.

### 2.2.1 FR model

Fruchterman and Reingold developed this model in 1991, following two main principles:

- drawing neighboring vertices near each other
- vertices should not be drawn too near to each other

The FR model uses the following stress-function:

$$S_{FR}(p(G)) = \frac{1}{6} \sum_{\{u,v\} \in E} \parallel p(u) - p(v) \parallel^3 - \sum_{\{u,v\} \in V^2} log \parallel p(u) - p(v) \parallel \tag{4}$$

Observe the following:

- the further apart neighboring vertices get mapped the bigger the stress
- small distances between vertices let the logarithm explode
- the logarithm dominates at small distances and hinders the graph from collapsing, or converging into a single point

$S_{FR}$ satisfies the two main principles, while being rather quick when implemented into an algorithm. The attractive force is only calculated over $F$ and the repulsive force over $W$, the authors name this as an advantage over Davidson and Harel's model, which also penalized edge crossing.

### 2.2.2 ERLL model

The Edge Repulsion LinLog model by Andreas Noack, 2004, places its focus on the graph's edges. For $v \in V$, let $deg(v)$ denote the number of edges $v$ is an endpoint of.

$$S_{ERLL}(p(G)) = \frac{1}{2} \sum_{\{u,v\} \in E} \parallel p(u) - p(v) \parallel - \sum_{\{u,v\} \in V^2} \deg(u) \deg(v) \log \parallel p(u) - p(v) \parallel \tag{5}$$

The edges cause both attraction and repulsion. Let us compare the ERLL with the FR model. Let $G$ be a graph. Now consider the unordered pairs $u, v \in V^2$. If $G$ is of uniform degrees, then FR and ERLL are of the same repulsion, only difference is a scalar $c = deg(v)^2$. Assume $G$ is of ununiform degrees, in FR the repulsion remains the same, but in ERLL vertices $u, v$ of high degree repel each other proportional to $deg(u)deg(v)$. The repulsive force in the FR model only depends on distance, letting "big nodes" be pulled inside the graph's embedding by the attractive force. The ERLL solves this issue, spacing "big nodes" evenly. This results in clearer embeddings of highly ununiform graphs.

# 3 Implementation

## 3.1 Plotting a graph embedding

To plot our graph embedding, we use the Graphs library [3] as well as the Plots library [4].

```
using Plots
using Graphs

function plot_graph(g::SimpleGraph, p::Matrix{Float64})
    plt = plot(legend = false, aspect_ratio = :equal)
    for e in edges(g)
        plot!(plt, [p[1, src(e)], p[1, dst(e)]], [p[2, src(e)], p[2, dst(e)]],
                color = :black)
    end
    scatter!(plt, p[1, :], p[2, :], markersize=5, color = :blue)
    return plt
end
```

Note that we first plot the edges by drawing straight lines between the vertices, and then we plot the vertices on top of the edges. This way, the vertices are not hidden behind the edges.

### 3.1.1 Higher dimension plotting

Plotting in higher dimensions is easy. Just extend the `plot!` and `scatter!` calls by one more argument for the third dimension:

```
function plot_graph_3d(g::SimpleGraph, p::Matrix{Float64})
    plt = plot(legend = false, aspect_ratio = :equal)
    for e in edges(g)
        plot!(plt, [p[1, src(e)], p[1, dst(e)]], [p[2, src(e)], p[2, dst(e)]], [p[3, src(e
                color = :black)
    end
```

```
    scatter!(plt, p[1, :], p[2, :], p[3, :], markersize=5, color = :blue)
    return plt
end
```

In the initialization of all the vectors and matrices, make sure to update the dimensions to 3.

### 3.2 Stress functions

Since the two stress functions explored in the theoretical part is structured in a similar way, we can implement them in a single function. We just need to pass the edge and vertex stress functions as arguments. To do this, we use anonymous functions:

```
function stress(
    g::SimpleGraph,
    edge_stress_f::Function,
    vertex_stress_f::Function,
    combine_f::Function)

    n = nv(g)
    edge_stress = 0
    for e in edges(g)
        edge_stress += edge_stress_f(src(e), dst(e))
    end
    vertex_stress = 0
    for i in 1:n
        for j in (i+1):n
            vertex_stress += vertex_stress_f(i, j)
        end
    end

    return combine_f(edge_stress, vertex_stress)
end
```

Implementing the FR and ERLL stress functions is then straightforward:

```
function fr_stress(g::SimpleGraph, p::Matrix{Float64})
    D = pairwise(Euclidean(), p, dims=2)
    edge_stress_f = (i, j) -> D[i, j]^3
    vertex_stress_f = (i, j) -> log(D[i, j] + 10^(-6))
    combine_f = (x, y) -> x / 6 - y
    return stress(g, edge_stress_f, vertex_stress_f, combine_f)
end
```

```julia
function erll_stress(g::SimpleGraph, p::Matrix{Float64}, deg::Vector{Int})
    D = pairwise(Euclidean(), p, dims=2)
    edge_stress_f = (i, j) -> D[i, j]
    vertex_stress_f = (i, j) -> deg[i] * deg[j] * log(D[i, j] + 10^(-6))
    combine_f = (x, y) -> x / 2 - y
    return stress(g, edge_stress_f, vertex_stress_f, combine_f)
end
```

Note that we add a small constant to the distance in the logarithm to prevent it from exploding when two vertices are mapped to the same point.

We use the `pairwise` function from the Distances library [1] to compute the distance matrix $D$. This is way more efficient than computing the distances in a nested loop.

Also note that the erll stress function takes an additional argument, which is the degree of each vertex. This is used in the calculation, but since the degree of each vertex does not change, it would be inefficient to calculate it inside the vertex stress function.

## 3.3 Gradient descent

The final piece in the puzzle is to actually use the plotting and the stress functions to find good embeddings of a given graph. To do this, we use gradient descent. We use the Zygote library [2] to compute the gradients of the stress functions.

With zygote, we can easily differentiate through our stress functions, which are implemented in a very straightforward way. Zygote does this by using source-to-source automatic differentiation, which means that it transforms our code into a form that can be differentiated, and then applies the chain rule to compute the gradients. This allows us to compute the gradients of our stress functions without having to manually derive them, which can be quite tedious and error-prone.

The principle of gradient descent is the following:

We initialize the positions of the vertices randomly in a 2D space. We then compute the stress and its gradient at each step, and update the positions by some rule. To check for convergence, we either stop after the maximum number of steps has been reached, or if the stress values do not change meaningfully. It is important that we do not check for equality of the stress values, since they are floating-point numbers and can be very close to each other without being exactly equal. Instead, we check if the absolute difference is smaller than a small threshold.

### 3.3.1 Simple gradient descent

This is the simple version of the gradient descent algorithm. One can imagine what this algorithm does by imagining a hiker being very tired and as such, trying to find the nearest town. To do that, he always goes in the direction of steepest descent. Crucially, he does not have any momentum.

```julia
function simple_gradient_descent(g::SimpleGraph, alpha::Float64, max_steps::Int64)
    n = nv(g) # Number of vertices
    deg = degree(g) # The degrees of the edges
    p = rand(2, n) # Initial positions

    stresses = Float64[]

    for k in 1:max_steps
        s, (grad,) = Zygote.withgradient(q -> fr_stress(g, q), p)
        push!(stresses, s)

        if length(stresses) > 1 && abs(stresses[end] - stresses[end-1]) < 1e-6
            println("converged!")
            println("iteration ", k)
            plot1 = plot_graph(g, p)
            plot2 = plot(stresses, label="stress")
            final = plot(plot1, plot2, layout=2)
            savefig(final, "converged.svg")
            break
        end

        p -= alpha * grad

        if k % 30 == 0
            plot1 = plot_graph(g, p)
            plot2 = plot(stresses, label="stress")
            display(plot(plot1, plot2, layout=2))
        end
    end
end
```

### 3.3.2 Nesterov's accelerated gradient descent

In this version of gradient descent, we not only go a small step in the direction of the gradient, but we also add momentum to the term.

This can be imagined by a ball rolling, which in addition to going downhill, also has some inertia, which makes it roll faster and faster as it goes down the hill. This can help to escape local minima and find better solutions.

Friction is modelled by $\beta$, which says how much of the previous momentum we keep. Because $\beta = 0.9$, we keep 90% of the previous momentum.

```julia
function nesterov_gradient_descent(g::SimpleGraph, alpha::Float64, beta::Float64, max_step
```

```julia
    n = nv(g) # Number of vertices
    deg = degree(g) # The degrees of the edges
    p = rand(2, n) # Initial positions

    stresses = Float64[]

    momentum = zeros(2, n)

    for k in 0:max_steps
        s, (grad,) = Zygote.withgradient(q -> fr_stress(g, q), p + beta * momentum)
        new_momentum = beta * momentum - alpha * grad
        push!(stresses, s)

        if length(stresses) > 50 && abs(stresses[end] - stresses[end-1]) < 1e-9
            println("converged!")
            println("iteration ", k)
            plot1 = plot_graph(g, p)
            plot2 = plot(stresses, label="stress")
            final = plot(plot1, plot2, layout=2)
            savefig(final, "converged.svg")
            break
        end

        p += new_momentum
        momentum = new_momentum

        if k % 30 == 0
            plot1 = plot_graph(g, p)
            plot2 = plot(stresses, label="stress")
            display(plot(plot1, plot2, layout=2))
        end
    end
end
```

# 4 Results

## 4.1 Figuring out good learning rates

### 4.1.1 Simple gradient descent, FR stress

Let's start with the simple gradient descent method, fr stress, $\alpha = 0.001$ and the wheel graph 11.

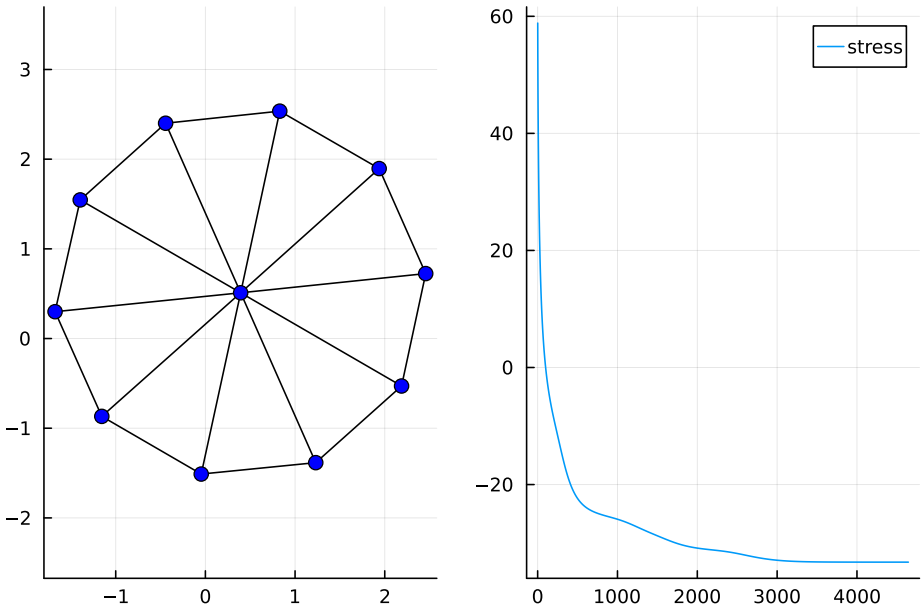


Figure 1: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.001$, Took 4644 Iterations

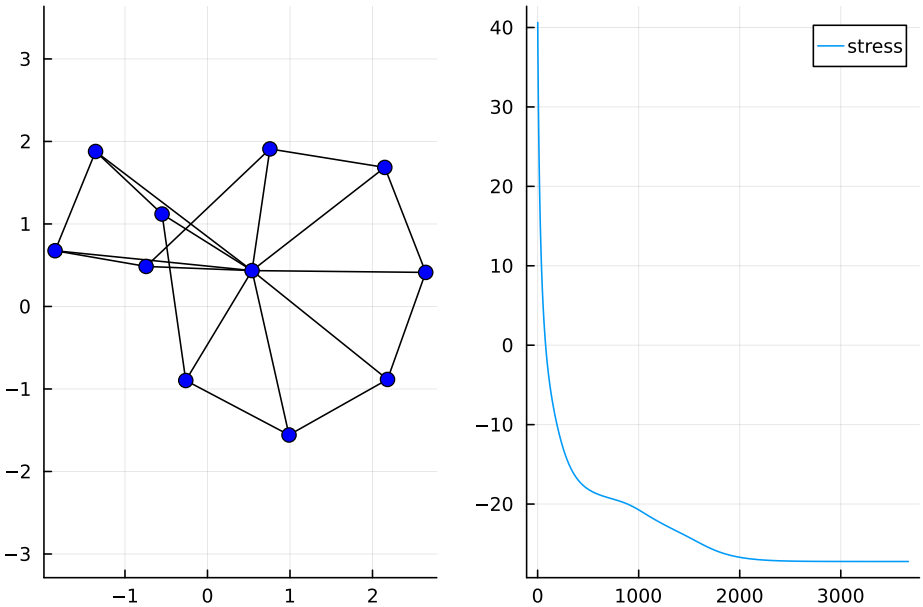


Figure 2: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.001$, Took 3669 Iterations

Almost always, the graph converges to something looking like Figure 1.
In a few cases (less than 10%), we converge into a local minimum like depicted in Figure 2

**Turning up the learning rate** When we increase the learning rate $\alpha$, there are interesting results.

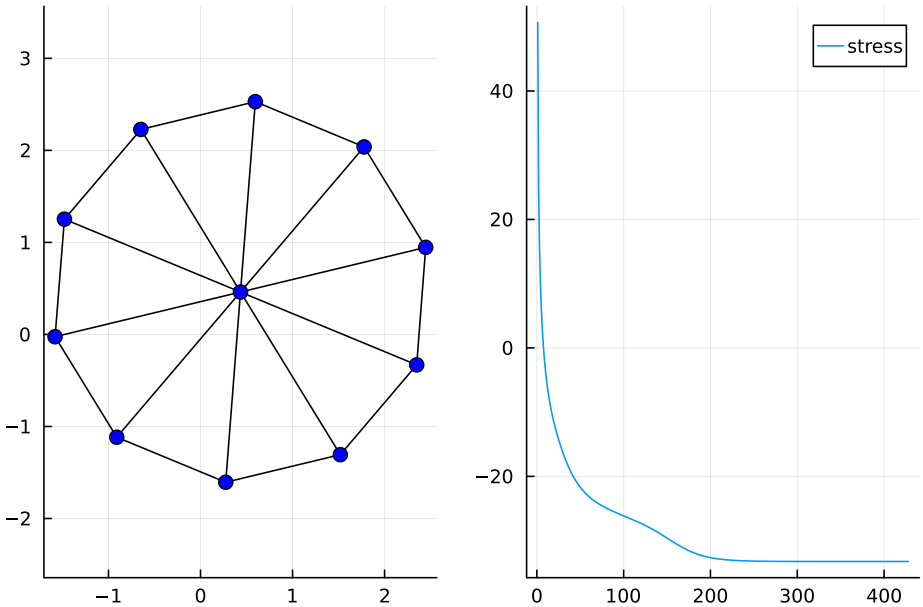


Figure 3: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.01$, Took 428 Iterations

With a learning rate $\alpha = 0.01$, the convergence speed dramatically increases with no visible poorer quality.

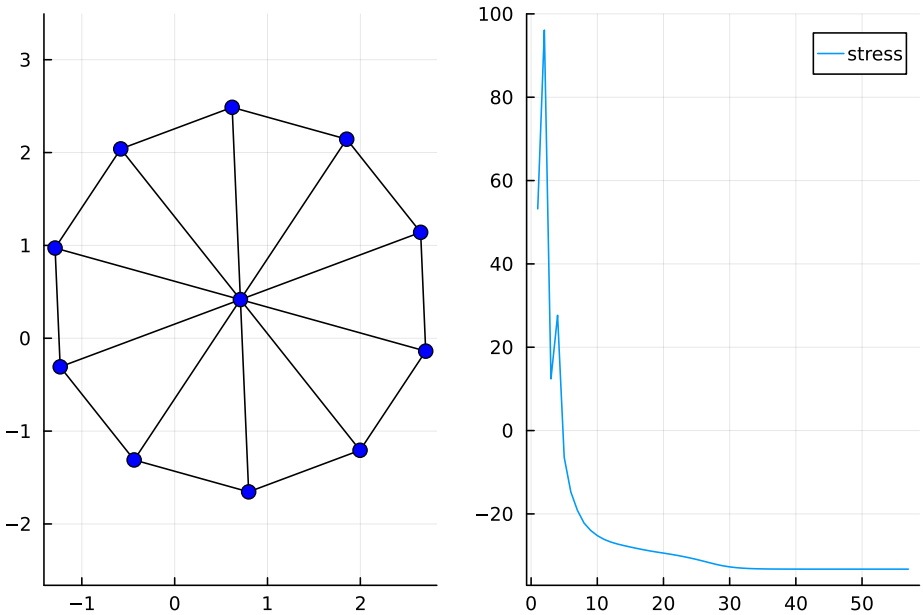


Figure 4: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.1$, Took 57 Iterations

At $\alpha = 0.1$, the convergence speed increases even more, with quality staying essentially the same. However, the rate of poorly converged graphs increases, however this is difficult to quantify. We also see that stress has actually increased twice, even though we were going "downhill". This is because we are taking too big steps, which makes us overshoot the "valley".

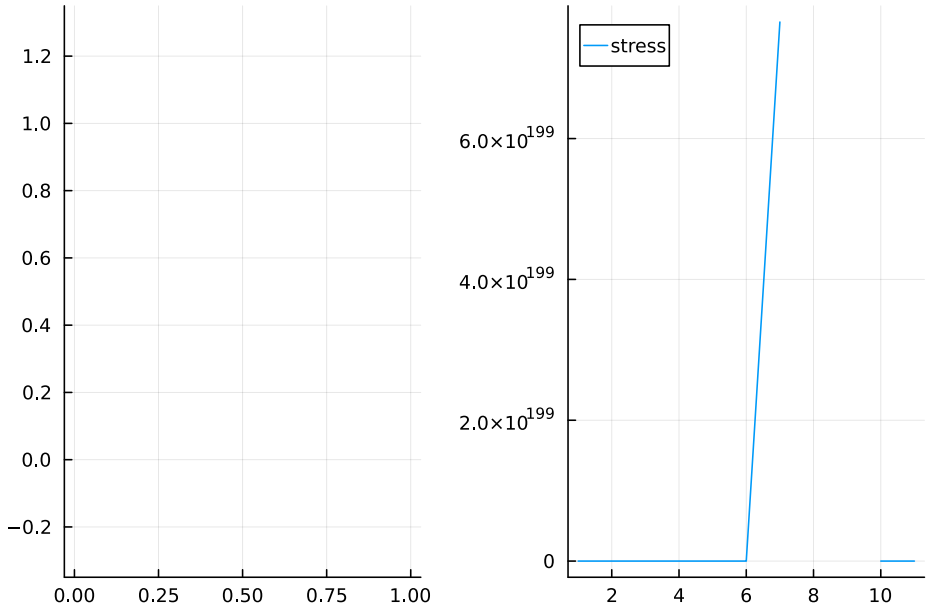


Figure 5: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.1$, Took 11 Iterations

Finally, at $\alpha = 1.0$, the algorithm does not converge at all, and the stress values explode. What's interesting is that it almost always takes 11 steps to diverge. Investigating what is going on reveals that most of the paramters turn to $NaN$ at step 10, not changing at step 11 and therefore "converging" as per our definition. The reason for this being essentially repeatable even though the initial positions are random is because the initial positions are bound to be in a range between 0 and 1.

From this, we can learn that we should always be careful when choosing the learning rate. For a simple graph, there are not as many local minima, so bigger learning rates still work, but we should not do this for a more complex graph.

### 4.1.2 Simple gradient descent, ERLL stress

Continuing with the simple gradient descent method, erll stress, $\alpha = 0.001$ and the wheel graph 11, we see that convergence is painfully slow. On the graph, we can see the verticies spreading out way more than with fr stress. This reveals a fundamental difference: ERLL stress punishes verticies being close way more than FR stress, which makes the learning rate proportionally smaller. We increase the learning rate to $\alpha = 0.1$ and see that convergence is much faster.

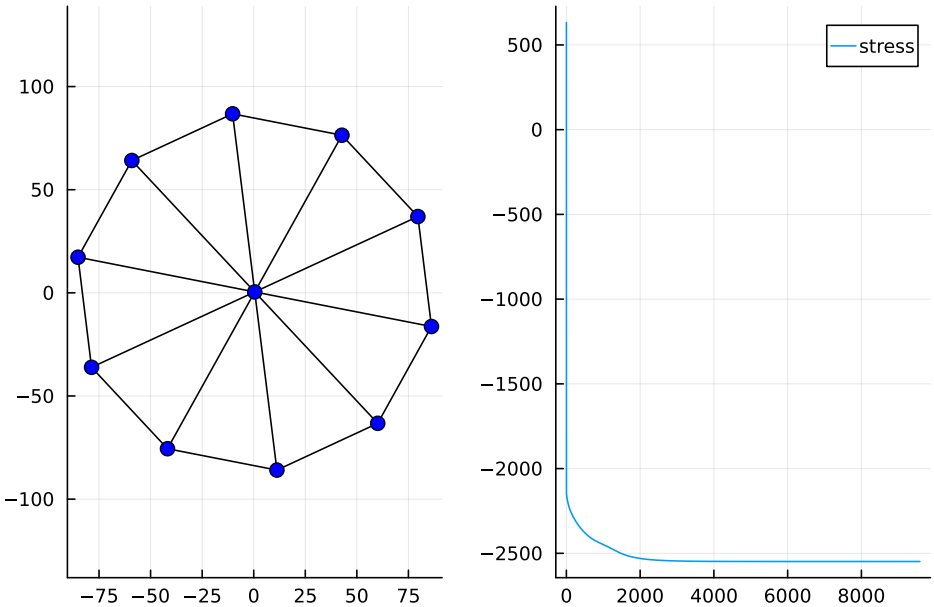


Figure 6: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.1$, Took 9586 Iterations

The function $erll_stress$ punishes verticies being close way more than $fr_stress$. For this reason, the graphs spreads out way more, which makes the learning rate proportionally smaller.

**A good learning rate** Considering that at $\alpha = 0.001$ compute times are reasonable for FR Stress, and at $\alpha = 0.1$ compute times are reasonable for ERLL Stress, we can say that these are good learning rates for the respective stress functions.

For simplicity reasons, we will use the same $\alpha$ for the nesterov gradient descent, one could of course tune this seperately.

## 4.2 Some graphs

In the following, we will show some examples of graphs drawn with the nesterov gradient descent and the different stress functions. They offer a comparison between the development of the stress functions. We will use $\alpha = 0.001$ for the fr stress and $\alpha = 0.1$ for the erll stress, as we have established these are good learning rates. As a momentum decay rate, we will use $\beta = 0.9$.

Because some graphs move very minimally even with this $\alpha$, we sometimes increase the convergence threshold $\epsilon$.

### 4.2.1 Icosahedral

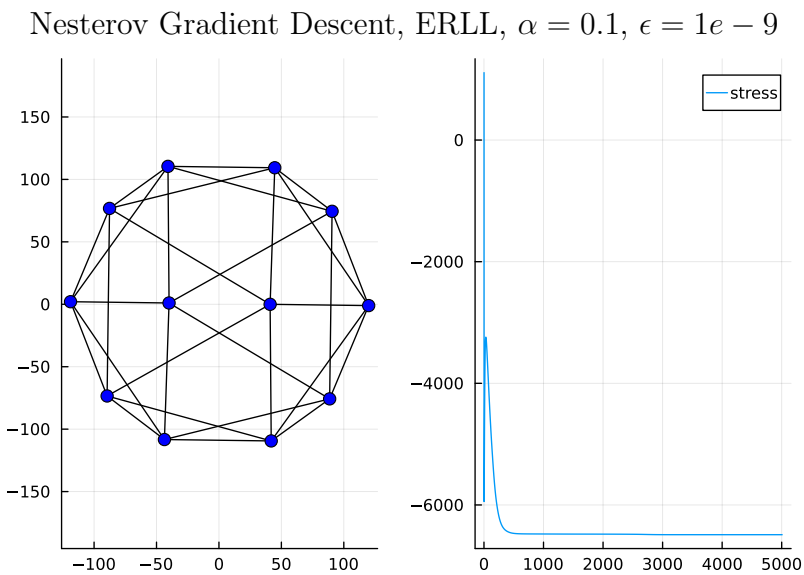


Figure 7: Icosahedral Graph, Took 5007 Iterations

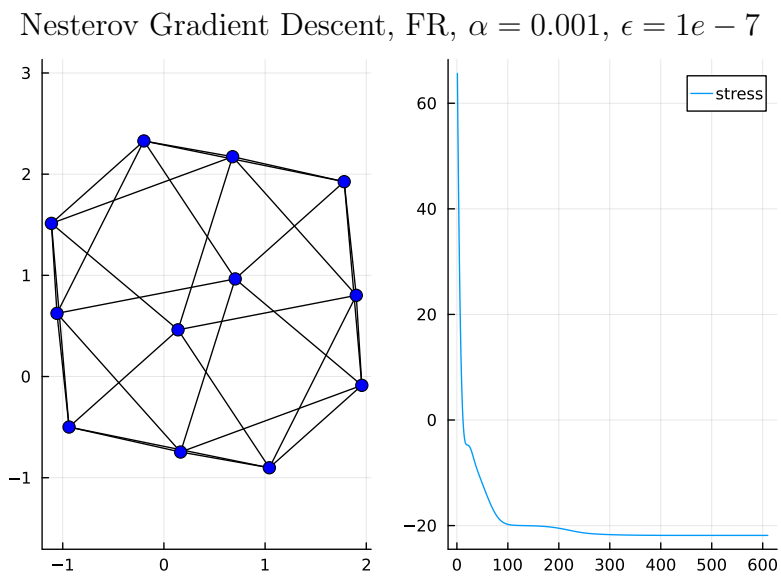


Figure 8: Icosahedral Graph, Took 609 Iterations

### 4.2.2 Karate network

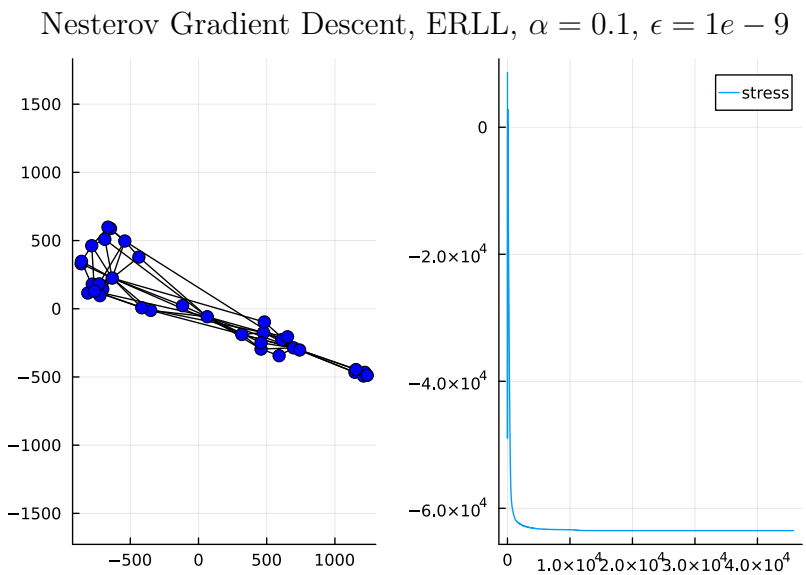


Figure 9: Karate Network, Took 45773 Iterations

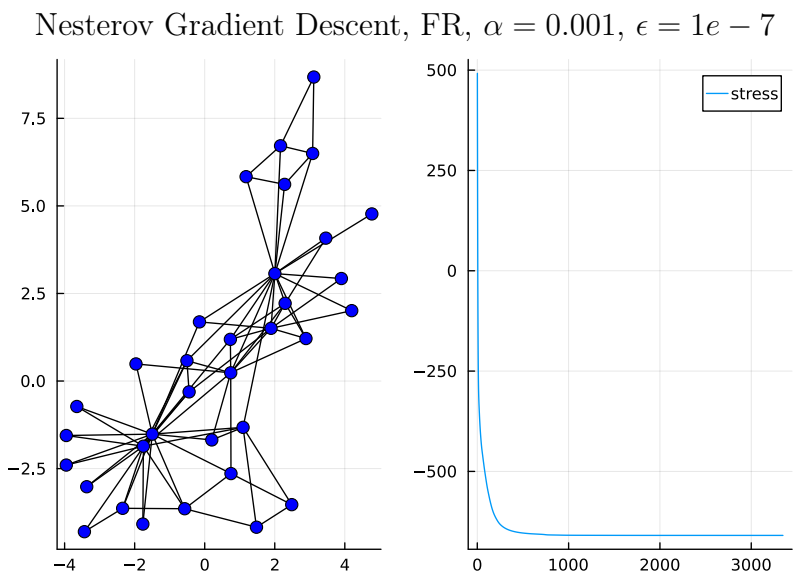


Figure 10: Karate Network, Took 3346 Iterations

### 4.2.3 Dorogovtsev Mendes Graph

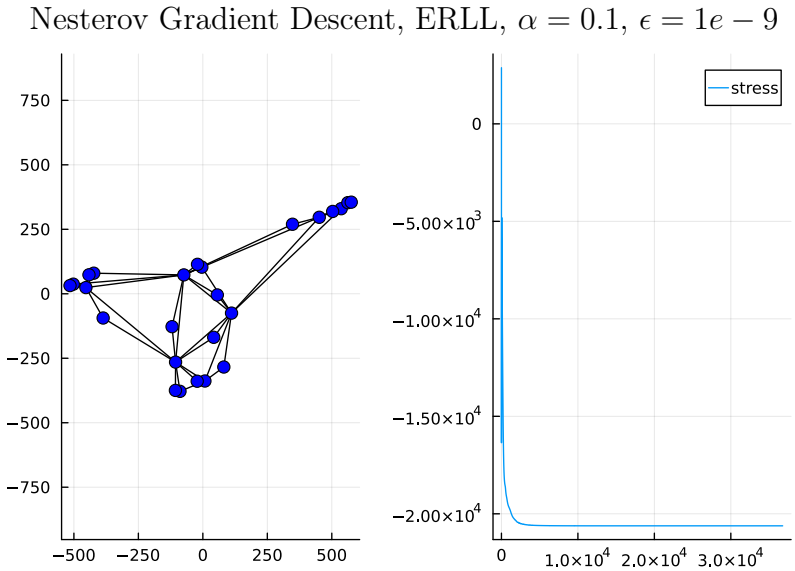


Figure 11: Dorogovtsev Mendes Graph, Took 36780 Iterations

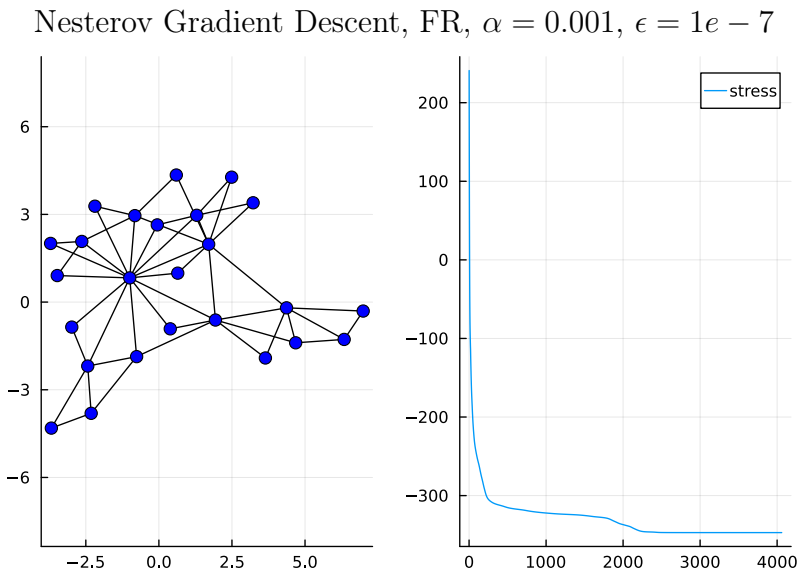


Figure 12: Dorogovtsev Mendes Graph, Took 4059 Iterations

### 4.2.4 Circular Ladder Graph

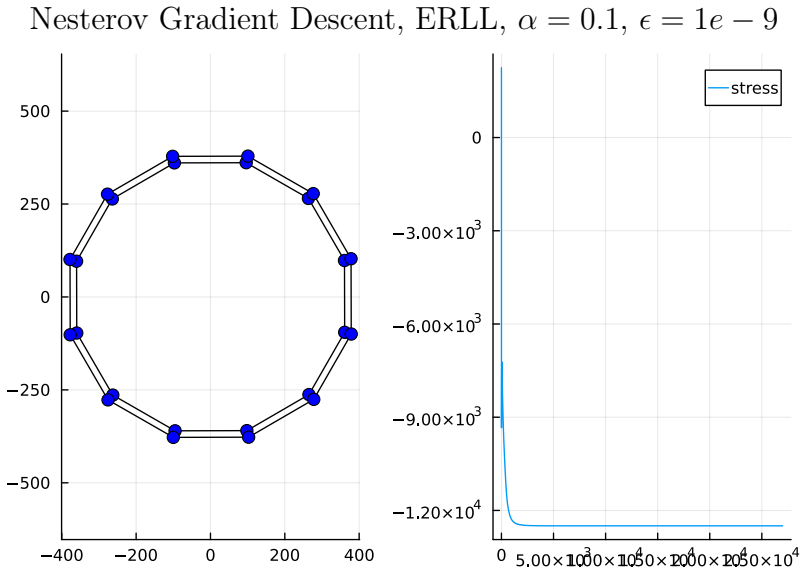


Figure 13: Circular Ladder Graph, Took (exactly) 27000 Iterations

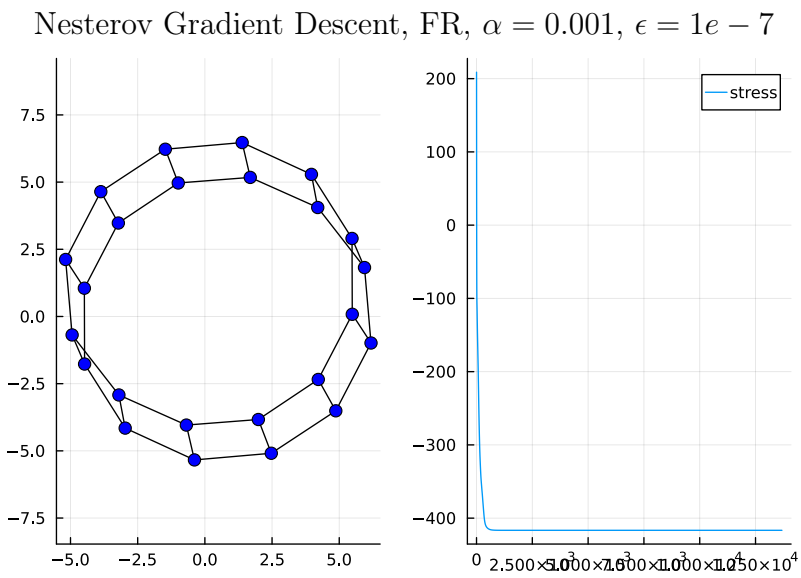


Figure 14: Circular Ladder Graph, Took 13683 Iterations

### 4.2.5 Watts Strogatz Graph

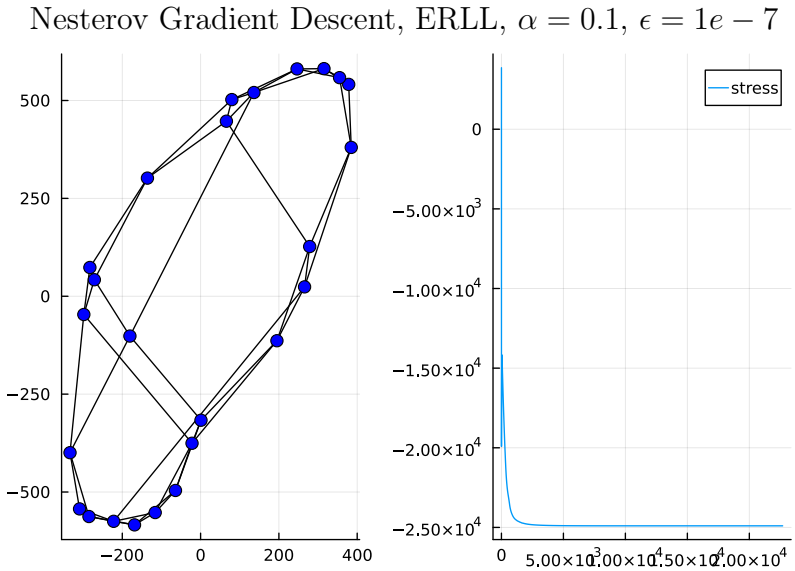


Figure 15: Watts Strogatz Graph, Took 22689 Iterations

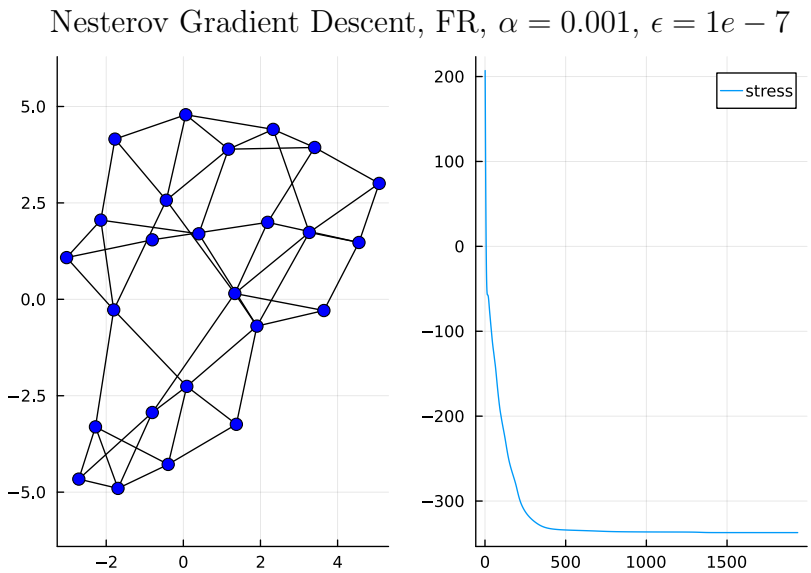


Figure 16: Watts Strogatz Graph, Took 1937 Iterations

### 4.2.6 Static Scale Free Graph

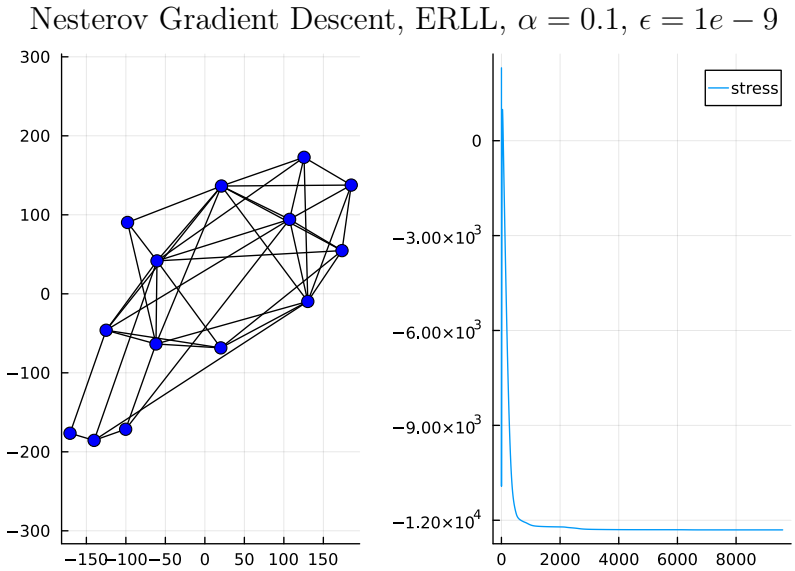


Figure 17: Static Scale Free Graph, Took 9587 Iterations

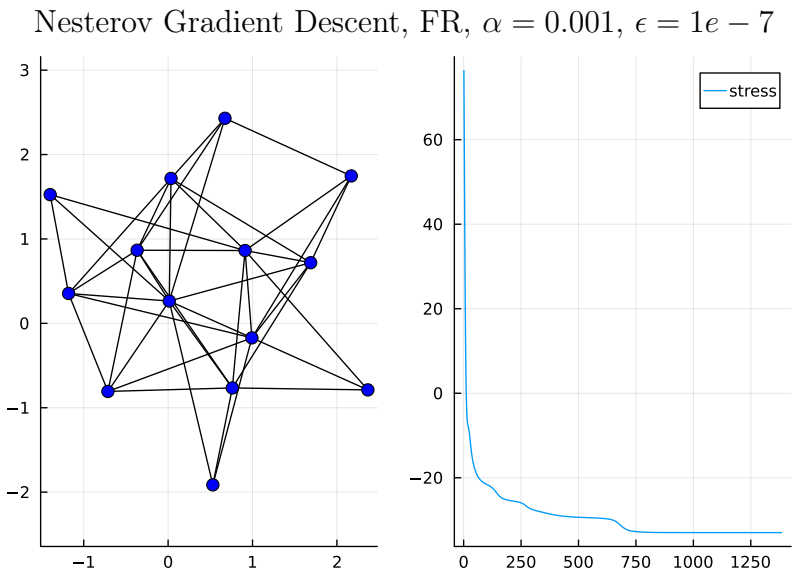


Figure 18: Static Scale Free Graph, Took 1383 Iterations

### 4.3 Conclusion

In this project, we have explained the theory behind graph drawing, implemented the graph drawing algorithm in Julia and compared different approaches and settings.

In our implementation, FR stress is not only much faster, but also produces better results. ERLL stress just pushes verticies too far apart.

With the settings above, we have created some nice drawings of graphs. They offer true insight into the structure even with no knowledge of the graphs themselves.

However, some graphs like the circular ladder graph almost always get stuck in suboptimal positions, and graphs like bipartite graphs get drawn in a very unsual way with this method.

While fun, for a truly good graph drawing algorithm, many improvements are possible.

## 5 References


[1] Dahua Lin and contributors. *Distances.jl: A Julia Package for Evaluating Distances (Metrics) Between Vectors*. Julia package, JuliaStats organization. 2014. URL: `https://github.com/JuliaStats/Distances.jl`.

[2] Michael Innes. "Don't Unroll Adjoint: Differentiating SSA-Form Programs". In: *CoRR* abs/1810.07951 (2018). arXiv: `1810.07951`. URL: `http://arxiv.org/abs/1810.07951`.

[3] James Fairbanks et al. *JuliaGraphs/Graphs.jl: an optimized graphs package for the Julia programming language*. 2021. URL: `https://github.com/JuliaGraphs/Graphs.jl/`.

[4] Simon Christ et al. "Plots.jl – A User Extendable Plotting API for the Julia Programming Language". In: *Journal of Open Research Software* (2023). DOI: `10.5334/jors.431`. URL: `https://openresearchsoftware.metajnl.com/articles/10.5334/jors.431`.